CHUYÊN ĐỀ BỒI DƯỠNG HỌC SINH GIỎI TIN HỌC

LÝ THUYẾT SỐ

qua 26 bài Codeforces

Lê Thành Nam

vòng tròn đồng dư: dây cung nối k với 2k (mod 240)

1	2	3	4	5	6	7	8	9	10	11	12
13	14	15	16	17	18	19	20	21	22	23	24
25	26	27	28	29	30	31	32	33	34	35	36
37	38	39	40	41	42	43	44	45	46	47	48
49	50	51	52	53	54	55	56	57	58	59	60
61	62	63	64	65	66	67	68	69	70	71	72
73	74	75	76	77	78	79	80	81	82	83	84
85	86	87	88	89	90	91	92	93	94	95	96
97	98	99	100	101	102	103	104	105	106	107	108
109	110	111	112	113	114	115	116	117	118	119	120

số nguyên tố · bội của 2 · bội của 3 · bội của 5 · bội của 7

Sàng Eratosthenes: mỗi hợp số được tô theo ước nguyên tố nhỏ nhất

---

Số nguyên tố · Ước số · UCLN · BCNN · Đồng dư · Euclid mở rộng · Tổ hợp modulo

Ngây thơ · Cải tiến · Lóe sáng · Tối ưu

LÊ THÀNH NAM

MỤC LỤC

Giới thiệu **2**

Chương 1. Số nguyên tố và sàng Eratosthenes **5**

Bài 1. Số gần nguyên tố (Codeforces 26A) 6
Bài 2. Bài toán Noldbach (Codeforces 17A) 13
Bài 3. Số T-nguyên tố (Codeforces 230B) 19
Bài 4. Ma trận nguyên tố (Codeforces 271B) 24

Chương 2. Phân tích thừa số, ước số, sàng ước nhỏ nhất **30**

Bài 5. Trừ ước (Codeforces 1076B) 31
Bài 6. Tích ba số (Codeforces 1294C) 35
Bài 7. Thử thách số dễ (Codeforces 236B) 40
Bài 8. Khôi phục mảng (Codeforces 1176D) 45
Bài 9. Hai ước số (Codeforces 1366D) 50

Chương 3. Ước chung lớn nhất và bội chung nhỏ nhất **55**

Bài 10. UCLN phức tạp (Codeforces 664A) 56
Bài 11. Ước chung (Codeforces 1203C) 60
Bài 12. UCLN theo hàng (Codeforces 1458A) 64
Bài 13. Orac và BCNN (Codeforces 1349A) 68

Chương 4. Đồng dư, lũy thừa nhanh, Fermat, Legendre **72**

Bài 14. Lũy thừa theo modulo (Codeforces 913A) 73
Bài 15. Số nguyên tố và phép nhân (Codeforces 1228C) 77
Bài 16. Tích các ước (Codeforces 615D) 82
Bài 17. Chữ số 0 tận cùng (Codeforces 1141C) 86

Chương 5. Euclid mở rộng và phương trình Diophantine **90**

Bài 18. Ebony và Ivory (Codeforces 633A) 91
Bài 19. Đường thẳng (Codeforces 7C) 95
Bài 20. Mùa giải bóng đá (Codeforces 1244C) 99

Chương 6. Tổ hợp theo modulo **103**

Bài 21. Số đẹp (Codeforces 300C) 104
Bài 22. Đếm mảng (Codeforces 1312D) 109

Chương 7. Đề kiểm tra tổng hợp **113**

Bài 23. Trừ số nguyên tố (Codeforces 1238A) 114
Bài 24. Fadi và BCNN (Codeforces 1285C) 117
Bài 25. Cùng UCLN (Codeforces 1295D) 121
Bài 26. Nỗi ám ảnh thú vị (Codeforces 869C) 125

Lê Thành Nam

GIỚI THIỆU

Cuốn sách này dành cho ai

Sách gồm 26 bài lý thuyết số chọn từ kho đề Codeforces (codeforces.com/problemset), xếp thành sáu chương theo mạch kiến thức: số nguyên tố, ước số, UCLN – BCNN, đồng dư và lũy thừa, Euclid mở rộng, tổ hợp theo modulo, cộng thêm một đề kiểm tra cuối sách. Độ khó Codeforces trải từ 800 đến 2000: bài dễ nhất dành cho học sinh mới làm quen với số học trong lập trình, bài khó nhất ngang tầm đề thi học sinh giỏi cấp tỉnh và vòng loại quốc gia.

Mỗi bài gồm hai phần. **Đề bài** được viết lại bằng tiếng Việt, giữ đúng yêu cầu, ràng buộc, định dạng vào/ra và ví dụ của đề gốc, kèm **bảng subtask** do sách bổ sung. **Hướng dẫn giải** không đưa ngay lời giải tốt nhất mà đi qua nhiều **tầng**: lời giải ngây thơ, hai đến ba tầng cải tiến, rồi mới đến lời giải tối ưu. Mỗi tầng đều có code chạy được, có thời gian chạy đo thật và có hình minh hoạ.

> **Về cách viết đề**
> *Đề trong sách không phải bản dịch từng câu. Mỗi đề được viết lại theo cách hiểu tự nhiên của người Việt, bỏ những chi tiết kể chuyện không ảnh hưởng đến lời giải. Mọi điều kiện, giới hạn, định dạng vào/ra và test ví dụ được giữ nguyên. Mỗi bài ghi mã bài và đường dẫn gốc để học sinh mở đề và nộp bài.*

CÁCH GIẢI

Hướng dẫn giải của mỗi bài chia thành các phần có dấu màu riêng. Trong mỗi phần, lời giải đi từng bước nhỏ (Bước 1, Bước 2, …), và hầu hết các bước đều có hình minh hoạ.

Phần	Nội dung
HIỂU ĐỀ	Vẽ ra các trường hợp đúng, sai để hiểu chính xác yêu cầu; nêu các nhận xét đầu tiên
NGÂY THƠ	Tầng 1: ý tưởng tự nhiên nhất, chạy tay, code vét cạn và độ phức tạp
BẾ TẮC	Thời gian chạy đo thật của tầng 1, chỉ ra nguyên nhân chậm bằng hình, kết thúc bằng một câu hỏi then chốt
CẢI TIẾN 1, 2, …	Các tầng trung gian: mỗi tầng sửa đúng một điểm yếu của tầng trước, có code, thời gian đo thật và subtask vượt qua được
LÓE SÁNG	Tính chất toán học quyết định dẫn tới lời giải tối ưu, có chứng minh và hình minh hoạ
TỐI ƯU	Code tối ưu, độ phức tạp và bảng so sánh tất cả các tầng trên mọi subtask
LƯU Ý	Những chỗ dễ sai và phần mở rộng

Thời gian chạy trong sách được đo thật bằng trình biên dịch g++ 13 (tuỳ chọn -O2) trên máy chủ Linux, tính cả thời gian đọc dữ liệu. Trên máy chấm Codeforces các con số có thể khác đôi chút, nhưng tỉ lệ giữa các tầng là như nhau.

SUBTASK VÀ CÁC TẦNG LỜI GIẢI

Đề thi học sinh giỏi trong nước thường chia test thành nhiều **subtask** (nhóm test) với ràng buộc tăng dần. Vì vậy mỗi bài trong sách có một bảng subtask: subtask đầu dành cho lời giải ngây thơ, các subtask giữa dành cho các tầng cải tiến, subtask cuối là ràng buộc gốc của Codeforces. Với những bài mà đề gốc có ràng buộc nhỏ, sách **mở rộng** ràng buộc ở subtask cuối để lời giải tối ưu thật sự cần thiết.

Lê Thành Nam

Cuối mỗi bài có bảng so sánh: mỗi tầng chạy trên test lớn nhất của từng subtask, ghi thời gian thật và số điểm đạt được. Khi luyện thi, học sinh nên viết **ngay** tầng ngây thơ để lấy điểm subtask nhỏ và để có chương trình đối chiếu, rồi mới nâng cấp dần.

SỬ DỤNG SÁCH

Với mỗi bài, học sinh nên đọc đề, tự nghĩ ít nhất 20–30 phút rồi mới đọc hướng dẫn. Khi đọc hướng dẫn, nên dừng sau mỗi "Câu hỏi then chốt" và tự trả lời trước khi đọc tiếp.

Mọi code trong sách đã được chạy với ví dụ của đề và đối chiếu chéo với nhau trên 120 test ngẫu nhiên mỗi bài (các bài có nhiều đáp án được kiểm tra bằng trình chấm riêng). Code tối ưu đã được đo trên test lớn nhất theo ràng buộc.

Nộp bài trên Codeforces

- Đăng kí tài khoản tại codeforces.com (mục **Register**).
- Mở đề theo đường dẫn trong sách, chọn **Submit Code**, chọn ngôn ngữ GNU G++17 hoặc G++20, dán code và nộp.
- Chương trình đọc từ bàn phím (stdin) và ghi ra màn hình (stdout); không mở tệp.
- Kết quả **Accepted** nghĩa là bài đúng. Các kết quả khác: Wrong answer (sai đáp án), Time limit exceeded (quá thời gian), Memory limit exceeded (quá bộ nhớ). Codeforces cho biết bài sai ở test thứ mấy.

04 bước tư duy trước một bài số học



Hình 26.4. Bốn bước xây dựng lời giải một bài số học

- **Bước 1. Đọc ràng buộc.** Ràng buộc quyết định độ phức tạp được phép (bảng dưới). Đây là "la bàn" cho cả lời giải.
- **Bước 2. Viết vét cạn đúng.** Làm đúng như đề mô tả. Tầng này lấy điểm subtask nhỏ và là chương trình đối chiếu cho các tầng sau.
- **Bước 3. Tìm chỗ lặp lại.** Đo thời gian, xem phần nào tính đi tính lại những việc giống nhau: kiểm tra nguyên tố từng số, đếm ước từng số, duyệt cặp không cần thiết.
- **Bước 4. Tìm tính chất toán học.** Một tính chất (ước đi theo cặp, gcd(a, b) = gcd(a, b − a), định lý Fermat, công thức Legendre, …) cho phép bỏ hẳn một vòng lặp hoặc tính trước cho mọi số cùng lúc.

Ràng buộc n	Độ phức tạp tối đa	Kĩ thuật thường gặp
≤ 20	$O(2^n \cdot n)$	vét cạn tập con
≤ 3 000	$O(n^2)$	duyệt cặp
≤ 10^5 – 10^6	$O(n\sqrt{n})$, $O(n \log n)$	thử chia, sàng spf

Lê Thành Nam

Ràng buộc n	Độ phức tạp tối đa	Kĩ thuật thường gặp
$\leq 10^7$	O(n log log n), O(n)	sàng Eratosthenes, sàng tuyến tính
$\leq 10^{12}$	O(√n)	thử chia tới √n
$\leq 10^{18}$	O(log n), $O(n^{1/3})$	lũy thừa nhanh, Euclid, Legendre

Bố cục sách

Chương	Chủ đề	Bài (mã Codeforces)
1	Số nguyên tố và sàng Eratosthenes	26A, 17A, 230B, 271B
2	Phân tích thừa số, ước số, sàng ước nhỏ nhất	1076B, 1294C, 236B, 1176D, 1366D
3	Ước chung lớn nhất và bội chung nhỏ nhất	664A, 1203C, 1458A, 1349A
4	Đồng dư, lũy thừa nhanh, Fermat, Legendre	913A, 1228C, 615D, 1114C
5	Euclid mở rộng và phương trình Diophantine	633A, 7C, 1244C
6	Tổ hợp theo modulo	300C, 1311D
7	Đề kiểm tra tổng hợp	1238A, 1285C, 1295D, 869C

Lê Thành Nam

Chương 1. Số nguyên tố và sàng Eratosthenes

> **Trọng tâm chương**
> Mọi bài trong chương xoay quanh câu hỏi "số này có phải số nguyên tố không?", hỏi cho **rất nhiều số** cùng lúc. Kĩ thuật chung là **chuyển từ kiểm tra từng số sang tính trước cho cả dãy**: thay hàm isPrime gọi đi gọi lại bằng một lần sàng Eratosthenes, rồi tra bảng trong O(1). Chương này cũng rèn thói quen đọc ràng buộc để chọn giữa O(√n) cho mỗi số và O(n log log n) cho cả dãy.

Kiến thức cần nhớ

- **Định nghĩa.** Số nguyên tố là số tự nhiên lớn hơn 1 chỉ có đúng hai ước dương là 1 và chính nó. Số 1 không phải số nguyên tố; 2 là số nguyên tố chẵn duy nhất.
- **Kiểm tra nguyên tố trong O(√n).** Nếu n là hợp số thì n có một ước d với $2 \le d \le \sqrt{n}$. Vì vậy chỉ cần thử chia n cho các d thỏa $d \cdot d \le n$. Nên viết điều kiện vòng lặp là `i * i <= n` (dùng kiểu long long) thay cho `i <= sqrt(n)` để tránh sai số số thực.
- **Sàng Eratosthenes.** Đánh dấu mọi số từ 2 đến N là nguyên tố; với mỗi i còn được đánh dấu, gạch các bội $i \cdot i$, $i \cdot i + i$, … Độ phức tạp O(N log log N), bộ nhớ O(N). Sàng tới 10^7 chạy dưới 0,1 giây.
- **Biến thể của sàng.** Thay vì chỉ đánh dấu đúng/sai, có thể cộng dồn thông tin vào các bội: đếm số ước nguyên tố phân biệt (cnt[x]++), lưu ước nguyên tố nhỏ nhất, tính trước "số nguyên tố gần nhất phía sau".
- **Tính chất hay dùng.** Số có đúng 3 ước khi và chỉ khi nó là bình phương của một số nguyên tố. Mọi số nguyên lớn hơn 1 đều có ít nhất một ước nguyên tố.

Các bài trong chương

Bài	Tên bài	Mã Codeforces	Độ khó	Số tầng lời giải
1	Số gần nguyên tố (Almost Prime)	26A	900	4
2	Bài toán Noldbach (Noldbach problem)	17A	1000	4
3	Số T-nguyên tố (T-primes)	230B	1300	4
4	Ma trận nguyên tố (Prime Matrix)	271B	1300	4

Lê Thành Nam

Bài 1. Số gần nguyên tố

Codeforces 26A - Almost Prime Thời gian: 2 giây · Bộ nhớ: 256 MB · Độ khó: 900
Đề gốc: https://codeforces.com/problemset/problem/26/A

Đề bài

Một số nguyên dương được gọi là **gần nguyên tố** nếu nó có **đúng hai** ước nguyên tố phân biệt. Chẳng hạn $6 = 2 \cdot 3$, $18 = 2 \cdot 3^2$ và $24 = 2^3 \cdot 3$ là các số gần nguyên tố; còn 4, 8, 9 (chỉ có một ước nguyên tố) và $42 = 2 \cdot 3 \cdot 7$ (có ba ước nguyên tố) thì không.

Cho số nguyên n, hãy đếm số lượng số gần nguyên tố trong đoạn từ 1 đến n (tính cả hai đầu).

Dữ liệu vào

Một dòng duy nhất chứa số nguyên n ($1 \le n \le 3000$).

Kết quả

Một số nguyên là số lượng số gần nguyên tố trong đoạn [1, n].

Ví dụ

Dữ liệu vào	Kết quả
10	2
21	

Giải thích ví dụ

Với n = 10 có hai số gần nguyên tố là 6 và 10. Với n = 21 có tám số: 6, 10, 12, 14, 15, 18, 20, 21.

Subtask

Các subtask dưới đây do sách bổ sung để luyện từng mức thuật toán; subtask cuối cùng là ràng buộc của đề gốc (hoặc mở rộng hơn đề gốc). Giới hạn thời gian 2,0 giây cho mọi subtask.

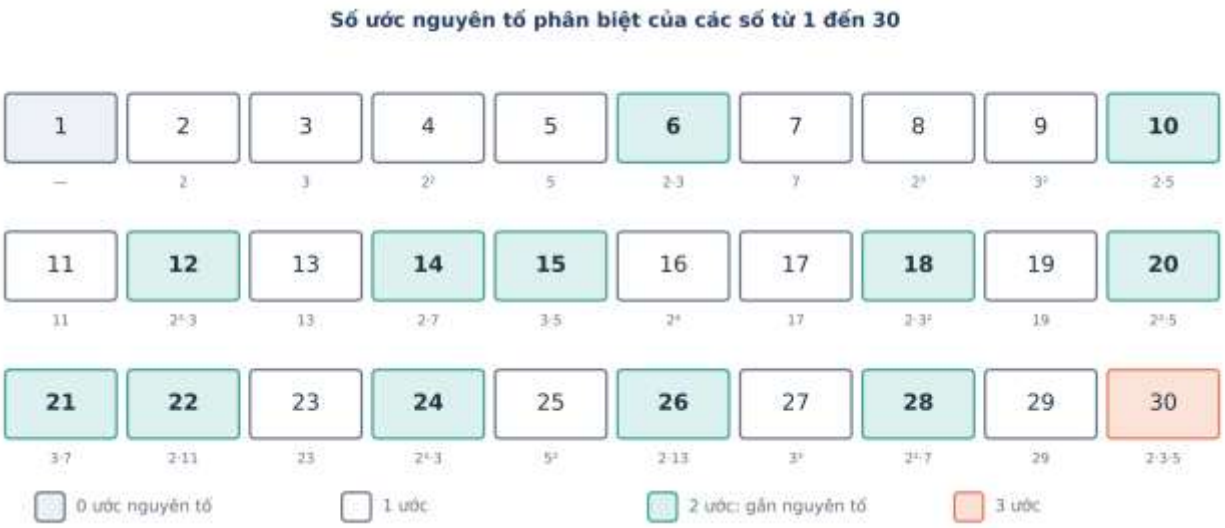
Subtask	Ràng buộc	Điểm
ST1	$n \le 3\,000$ (ràng buộc gốc)	30%
ST2	$n \le 10^5$	30%
ST3	$n \le 10^7$	40%

Hướng dẫn giải

HIỂU ĐỀ Hiểu đúng đề bài

Với mỗi số x, ta quan tâm đến **tập các ước nguyên tố khác nhau** của x, không quan tâm số mũ. Ví dụ $12 = 2^2 \cdot 3$ có tập ước nguyên tố {2, 3} (hai phần tử) nên 12 là số gần nguyên tố, dù 2 xuất hiện hai lần.

Lê Thành Nam



Hình 1.1. Số ước nguyên tố phân biệt của 1..30; ô xanh là số gần nguyên tố

Đếm các ô xanh trong hình: từ 1 đến 10 có 6, 10; từ 1 đến 21 có thêm 12, 14, 15, 18, 20, 21, đúng bằng hai ví dụ của đề (2 và 8). Hai nhận xét:

- Số 1 không có ước nguyên tố nào; mọi số nguyên tố p có đúng một ước nguyên tố. Các số này không bao giờ được đếm.
- Lũy thừa của một số nguyên tố (4, 8, 9, 16, 27, …) chỉ có **một** ước nguyên tố; 30 = 2·3·5 có **ba**. Cả hai đều không được đếm.

NGÂY THƠ Ý tưởng ngây thơ: thử mọi p ≤ x và kiểm tra nguyên tố

Cách nghĩ trực tiếp nhất: với mỗi x từ 1 đến n, **thử mọi p từ 2 đến x**; nếu p chia hết x và p là số nguyên tố thì tăng biến đếm. Để kiểm tra p nguyên tố, thử mọi ước d từ 2 đến p − 1.

Bước 1. Chạy tay với x = 12



Hình 1.2. Tầng 1 thử 11 giá trị p cho x = 12; chỉ hai ước 6 và 12 phải kiểm tra nguyên tố thêm

Bước 2. Cài đặt

Code. Tầng 1 – vét cạn

```
#include <bits/stdc++.h>
using namespace std;
bool isPrime(int p){                    // kiểm tra nguyên tố kiểu "ngây thơ": thử mọi d < p
    if (p < 2) return false;
    for (int d = 2; d < p; d++) if (p % d == 0) return false;
    return true;
}
```

Lê Thanh Nam

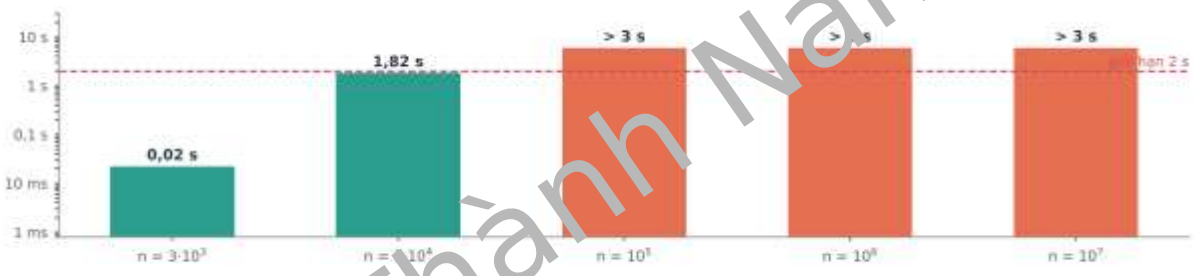
```
int main(){
    int n; cin >> n;
    int ans = 0;
    for (int x = 1; x <= n; x++) {
        int cnt = 0;                    // đếm số p nguyên tố chia hết x
        for (int p = 2; p <= x; p++)
            if (x % p == 0 && isPrime(p)) cnt++;
        if (cnt == 2) ans++;
    }
    cout << ans << "\n";
}
```

Bước 3. Đánh giá độ phức tạp

Vòng p chạy x − 1 lần cho mỗi x nên riêng hai vòng lặp ngoài đã tốn 1 + 2 + … + n ≈ $n^2/2$ phép chia. Mỗi lần p chia hết x, hàm isPrime(p) tốn thêm tới p phép chia; cộng theo mọi x, phần này cũng cỡ n^2. Tổng cộng **O(n^2)**: đủ cho ràng buộc gốc n ≤ 3000 (khoảng $9 \cdot 10^6$ phép tính).

BẾ TẮC Bế tắc: vì sao vét cạn không dùng được khi n lớn

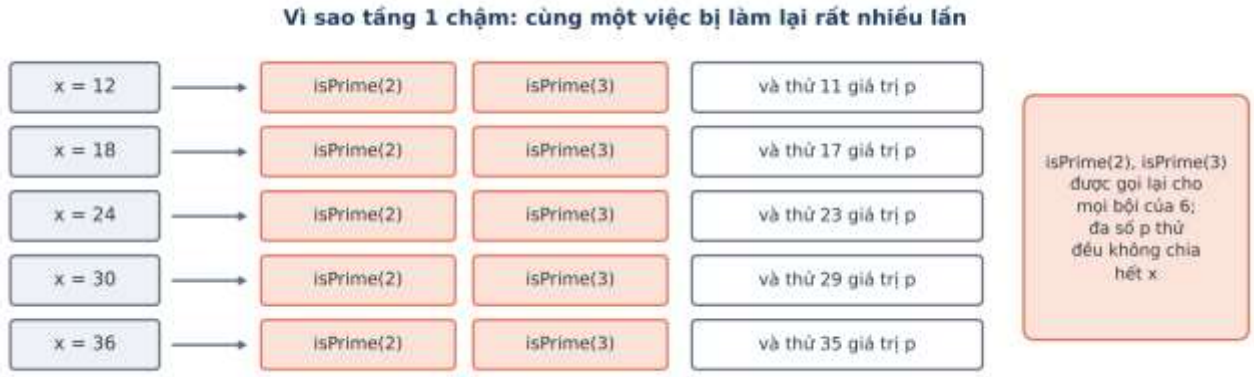
Bước 1. Đo thời gian thật



Hình 1.3. Thời gian chạy của tầng 1: n tăng 10 lần thì thời gian tăng khoảng 100 lần

Với n = 3000, tầng 1 chạy chưa tới 0,03 giây. Nhưng mỗi khi n tăng 10 lần, thời gian tăng khoảng 100 lần: n = $3 \cdot 10^4$ đã gần 2 giây, n = 10^5 vượt giới hạn, còn n = 10^7 cần khoảng 10^{14} phép tính (nhiều ngày).

Bước 2. Tìm nguyên nhân



Hình 1.4. Tầng 1 lặp lại cùng một việc: thử p không chia hết x và kiểm tra lại cùng một số nguyên tố

- **Thử thừa:** với x = 30 ta thử 29 giá trị p, nhưng chỉ 7 giá trị chia hết 30.

Lê Thành Nam

- **Lặp lại:** isPrime(2) được gọi lại cho mọi số chẵn, isPrime(3) cho mọi bội của 3, … Kết quả luôn như nhau.

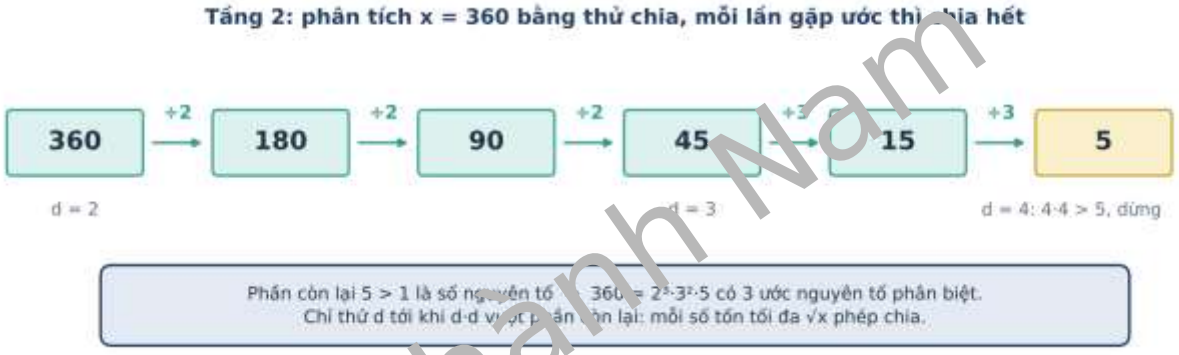
> **Câu hỏi then chốt**
> Có cách nào tìm ra các ước nguyên tố của x mà **không phải thử mọi p ≤ x**, và không phải kiểm tra nguyên tố từng ước?

CẢI TIẾN 1 Cải tiến 1: phân tích thừa số bằng thử chia tới √x

Bước 1. Ý tưởng

Thử d = 2, 3, 4, … Mỗi khi d chia hết x thì **chia x cho d tới khi không chia hết nữa** và tăng biến đếm. Khi đó d chắc chắn là số nguyên tố: mọi ước nhỏ hơn của d đã bị chia hết khỏi x trước đó, nên một hợp số d không thể chia hết phần còn lại. Không cần hàm isPrime nữa.

Chỉ cần thử tới khi d·d vượt phần còn lại của x: nếu phần còn lại y > 1 không có ước nào ≤ √y thì y là số nguyên tố, và nó là ước nguyên tố cuối cùng.



Hình 1.5. Phân tích 360 = 2³·3²·5: chỉ cần thử d = 2, 3, 4

Code. Tầng 2 – thử chia tới √x

```
#include <bits/stdc++.h>
using namespace std;
int distinctPrimes(int x){              // phân tích thừa số bằng thử chia tới sqrt(x)
    int cnt = 0;
    for (int d = 2; d * d <= x; d++)
        if (x % d == 0) { cnt++; while (x % d == 0) x /= d; }
    if (x > 1) cnt++;                   // phần còn lại là một số nguyên tố
    return cnt;
}
int main(){
    int n; cin >> n;
    int ans = 0;
    for (int x = 1; x <= n; x++) if (distinctPrimes(x) == 2) ans++;
    cout << ans << "\n";
}
```

Bước 2. Kết quả

Mỗi số tốn không quá √x phép chia, tổng **O(n√n)**. Tầng 2 qua ST1 và ST2 (n = 10^5 chỉ mất 0,03 giây), nhưng n = 10^7 cần khoảng $10^7 \cdot 3 \cdot 10^3/3 \approx 10^{10}$ phép chia, vẫn quá giới hạn.

> **Câu hỏi then chốt**
> Hai số 12 và 24 có chung ước 2, 3 nhưng tầng 2 vẫn tìm lại từ đầu cho từng số. Có thể **tính trước** thông tin ước cho mọi số cùng lúc không?

Lê Thanh Nam

CẢI TIẾN 2 Cải tiến 2: tính trước ước nguyên tố nhỏ nhất bằng sàng

Bước 1. Sàng ước nguyên tố nhỏ nhất

Biến thể của sàng Eratosthenes: với mỗi i chưa được đánh dấu (tức i nguyên tố), gán spf[j] = i cho các bội j của i chưa có giá trị. Sau sàng, spf[x] là ước nguyên tố nhỏ nhất của x.



Hình 1.6. *Mảng spf và cách phân tích 18 trong ba bước tra mảng*

Bước 2. Phân tích mọi x bằng spf

Lặp: p = spf[y], tăng biến đếm, chia y cho p tới khi hết. Mỗi bước tra mảng O(1) và mỗi lần chia làm y giảm ít nhất một nửa, nên phân tích x tốn O(log x).

Code. Tầng 3 – sàng spf rồi phân tích

```
#include <bits/stdc++.h>
using namespace std;
int main(){
    int n; cin >> n;
    vector<int> spf(n + 1, 0);          // spf[x] = ước nguyên tố nhỏ nhất của x
    for (int i = 2; i <= n; i++)
        if (spf[i] == 0)
            for (long long j = i; j <= n; j += i) if (spf[j] == 0) spf[j] = i;
    int ans = 0;
    for (int x = 2; x <= n; x++) {      // phân tích x bằng spf: O(log x)
        int y = x, cnt = 0;
        while (y > 1) { int p = spf[y]; cnt++; while (y % p == 0) y /= p; }
        if (cnt == 2) ans++;
    }
    cout << ans << "\n";
}
```

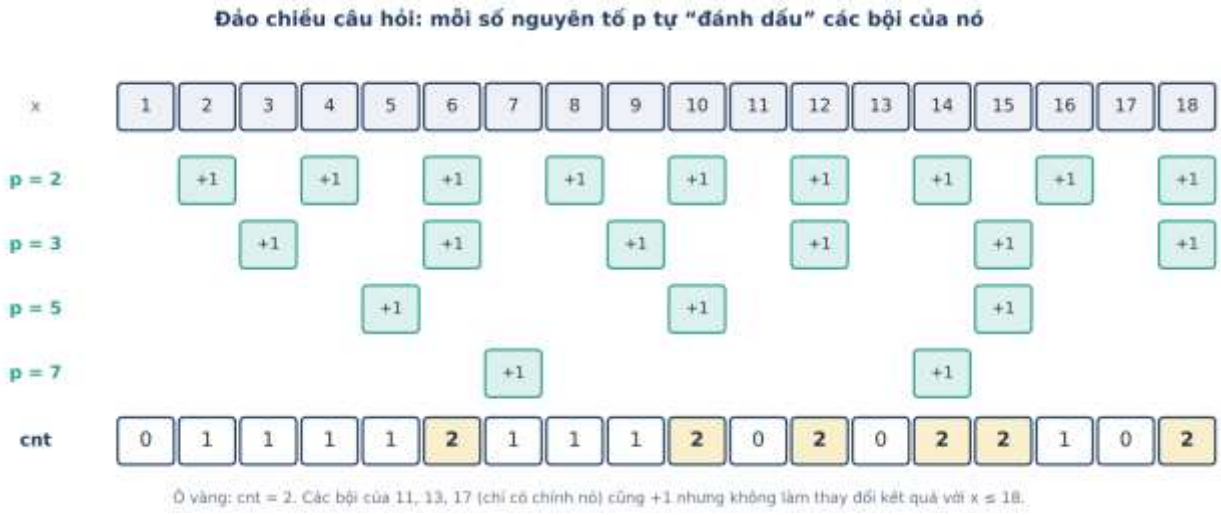
Sàng tốn O(n log log n), phân tích tốn O(n log n). Với $n = 10^7$, tầng 3 chạy khoảng 1,3 giây: kịp giới hạn 2 giây của đề nhưng cần mảng spf 40 MB và vẫn còn bước phân tích thừa.

LÓE SÁNG Lóe sáng: đảo chiều câu hỏi

Bước 1. Từ "x có những ước nào?" sang "p là ước của những số nào?"

Ta không cần biết các ước nguyên tố **cụ thể** của x, chỉ cần biết **có bao nhiêu**. Mà một số nguyên tố p là ước của đúng các số p, 2p, 3p, … Vậy với mỗi số nguyên tố p, cộng 1 vào cnt của tất cả các bội của p. Cuối cùng cnt[x] chính là số ước nguyên tố phân biệt của x.

Lê Thành Nam



Hình 1.7. Mỗi số nguyên tố cộng 1 vào các bội của nó; cnt = 2 là số gần nguyên tố

Bước 2. Nhận biết p nguyên tố ngay trong lúc sàng

Duyệt p tăng dần. Khi tới p, mọi số nguyên tố nhỏ hơn p đã cộng xong vào các bội của chúng. Nếu cnt[p] vẫn bằng 0 thì không số nguyên tố nhỏ hơn nào chia hết p, tức **p là số nguyên tố**. Không cần mảng đánh dấu riêng.

TỐI ƯU Lời giải tối ưu

Bước 1. Cài đặt

Code. Tầng 4 – sàng đếm ước nguyên tố

```
#include <bits/stdc++.h>
using namespace std;
int main(){
    int n; cin >> n;
    // cnt[x] = số ước nguyên tố phân biệt của x
    vector<int> cnt(n + 1, 0);
    for (int p = 2; p <= n; p++)
        // p chưa bị số nào đánh dấu -> p nguyên tố
        if (cnt[p] == 0)
            for (int x = p; x <= n; x += p) cnt[x]++;
    int ans = 0;
    for (int x = 1; x <= n; x++) if (cnt[x] == 2) ans++;
    cout << ans << "\n";
}
```

Bước 2. Độ phức tạp

Số phép cộng là n/2 + n/3 + n/5 + n/7 + … (tổng theo các số nguyên tố p ≤ n), bằng **O(n log log n)**. Với $n = 10^7$, con số này chỉ khoảng $3 \cdot 10^7$.

Lê Thành Nam

Bước 3. So sánh các tầng

Tầng	Ý tưởng	Độ phức tạp	$n = 3 \cdot 10^3$ (ST1)	$n = 3 \cdot 10^4$ (ST2)	$n = 10^5$ (ST2)	$n = 10^6$ (ST3)	$n = 10^7$ (ST3)	Điểm
1	Vét cạn: thử mọi $p \le x$	$O(n^2)$	0,02 s	1,82 s	quá giờ	quá giờ	quá giờ	30
2	Thử chia tới $\sqrt{x}$	$O(n\sqrt{n})$	4,5 ms	0,01 s	0,03 s	0,56 s	quá giờ	60
3	Sàng spf + phân tích	$O(n \log n)$	4,7 ms	4,5 ms	9,1 ms	0,08 s	1,20 s	100
4	Sàng đếm ước nguyên tố	$O(n \log \log n)$	4,0 ms	3,7 ms	4,4 ms	0,02 s	0,38 s	100

Thời gian đo thật trên test lớn nhất của từng subtask; giới hạn 2,0 giây. Ô đỏ: quá giờ hoặc sai. Cột "Điểm" cộng phần trăm của các subtask mà tầng đó vượt qua.

LƯU Ý Những chỗ dễ sai

- Đếm nhầm ước có lặp: $12 = 2 \cdot 2 \cdot 3$ có **hai** ước nguyên tố phân biệt, không phải ba.
- Ở tầng 2, quên trường hợp phần còn lại $y > 1$ sau vòng lặp (ví dụ $14 = 2 \cdot 7$: vòng lặp dừng ở $d = 3$ và còn lại 7).
- Khi mở rộng n tới 10^7, mảng int tốn 40 MB; dùng vector<char> (ước không vượt quá 8 với $n \le 10^7$) giảm còn 10 MB.
- Mở rộng: cùng kĩ thuật "mỗi p cộng vào các bội" cho phép tính số ước, tổng ước, hàm Euler của mọi số $\le n$.

Lê Thành Nam